

Local Agents and Agentic RAG

1. Executive assessment

A useful, fully local agent and agentic retrieval-augmented generation system is feasible on a 64 GB Apple Silicon machine for one person, provided that model size, context, ingestion, and concurrent inference are bounded. The recommended first build uses native llama.cpp with Metal, a quantized Qwen instruction model, local embedding and reranking models, a Python application with LangGraph, and PostgreSQL with pgvector and native full-text search. This is an engineering recommendation supported by documented component capabilities; its speed and answer quality require measurement on the actual machine.^{[25][26][31][35][38]}

The hardware target is the supplied incoming Mac mini configuration: M5 Pro, 64 GB unified memory, 2 TB storage, and the supplied 20 CPU/20 GPU core counts. Capacity planning below uses memory and single-user operation rather than relying on those core counts. They are deployment inputs, not independently verified Apple specifications. The document workload is not yet specified; begin with text, Markdown, text PDFs, scanned PDFs, and DOCX, and measure a representative sample before bulk ingestion.

The central design decision is to establish a strong fixed retrieval baseline before adding adaptive search. An agent should perform extra work when it can identify a missing fact, a necessary second hop, or an unresolved contradiction. It should not be rewarded simply for creating more roles, generating longer reasoning, or continuing indefinitely. There is no universally ideal agent architecture; the appropriate choice depends on task dependencies, the retriever, model capability, reliability requirements, and the computation budget.^{[9][10][12]}

Google's June 2026 work motivates an independent local experiment. Its hosted-system accuracy is not an expected result for a quantized model on private documents.^[8]

The result should expose a browser interface privately through a VPN, while models, documents, indexes, prompts, and evaluation remain on the Mac. Remote access and air-gapped operation are separate deployment profiles: remote access requires networking, but does not require hosted model inference. Plain WireGuard is the strict self-managed option; Tailscale is an optional convenience with an external coordination service.^{[45][46][47]}

2. Definitions and conceptual foundations

An LLM is a conditional generator. An LLM agent is a surrounding program that repeatedly presents a task and current observations to a model, obtains a proposed next action, validates it, executes an authorized tool, records the result, and decides whether to continue. The model proposes; application code controls execution. A role prompt alone does not provide persistence, authorization, scheduling, tool execution, or a stopping rule.

Classical agents include reactive rules, planners, reinforcement-learning policies, and multi-agent simulations. The systems considered here use language models as some portion of the policy or planner. An autonomous agent has discretion within a defined envelope. Autonomy is a matter of

permitted decisions, resources, and side effects, rather than a binary property or evidence of general intelligence.

ReAct established a widely used pattern of interleaving reasoning and actions with observations from an environment. Its significance is the feedback loop, rather than any requirement to expose long private reasoning text. A practical implementation can retain action arguments, tool observations, evidence references, brief decision summaries, and outcomes without storing a model's full internal reasoning.^[2]

System	Who selects the next step?	Appropriate use
Fixed workflow	Application code	Repeatable ingestion, validation, exports, approvals
Tool-using agent	Model proposes; code validates	Choosing among search, lookup, calculation, and finish
Agentic RAG	Adaptive retrieval policy inside a bounded workflow	Missing evidence, cross-document hops, source selection
Multi-agent system	Several agent policies and a coordinator or messaging topology	Independent research branches or distinct permission domains
Scheduled agent	A scheduler starts a bounded run	Periodic local indexing or reports; scheduling is separate from reasoning

RAG combines generated language with external information. The original 2020 formulation combined a parametric sequence model with non-parametric retrieved memory. Modern practical RAG is broader and does not necessarily train the retriever and generator jointly.^[1] Fine-tuning changes model parameters; RAG supplies external evidence at inference time. Fine-tuning is useful for behavior or domain formats, but is a poor primary mechanism for frequently changing facts, exact quotations, deletion, or per-document access rules.

Agentic RAG adds decisions about retrieval: whether to search, what to search for, which collection or tool to use, how to reformulate a query, whether to expand a document, and whether the accumulated evidence is sufficient. It can use one agent. A graph database is optional. A pipeline with several named LLM stages is not necessarily adaptive, and a fixed workflow can already include query expansion, hybrid search, and reranking.^{[6][15]}

The field did not start in 2025. Self-RAG introduced trained retrieval and reflection behavior in 2023; Corrective RAG used retrieval assessment and corrective actions in 2024. A prompted critique stage is not a reproduction of Self-RAG's trained reflection-token mechanism. Similarly, replacing CRAG's web fallback with an internal search tool changes the evaluated method, although it can be a sensible local adaptation.^{[3][4]}

3. Research map and strength of evidence

The evidence cutoff is September 12, 2026. Dates below distinguish initial publication from later revisions where verified. Papers support findings in their stated experiments; repositories document available implementations; model cards usually report developer-run evaluations. None independently establishes end-to-end quality on a private corpus.

Established means a well-supported technique or documented capability, not guaranteed reliability. **Emerging** means promising results with limited transfer evidence. **Experimental** means substantial deployment uncertainty, training burden, or sensitivity to evaluation choices. Recommendations and acceptance thresholds in this document are engineering judgments, not published benchmark results.

Work and date	Main contribution	Evidence assessment and local consequence
RAG, 2020	Combines generated answers with retrieved external memory	Foundational; preserve the distinction between model knowledge and external evidence. ^[1]
ReAct, 2022 / ICLR 2023	Alternates reasoning, actions, and observations	Foundational agent pattern; host-side validation remains necessary. ^[2]
Self-RAG, 2023 / 2024	Learns retrieval and reflection tokens	Established research result; specialized training, not a generic prompt recipe. ^[3]
Corrective RAG, 2024	Evaluates retrieval quality and triggers correction	Useful precursor; public-web fallback is outside strict offline mode. ^[4]
GraphRAG, April 2024; revision February 2025	Entity graph and community summaries for global corpus questions	Useful for broad synthesis; extraction and summaries add cost and error paths. ^[5]
Agentic RAG survey, January 2025; v4 April 2026	Organizes agent cardinality, control, autonomy, and knowledge representation	Taxonomy, not an independent proof that agentic RAG always improves results. ^[6]
Sufficient Context, November 2024 / ICLR 2025	Separates relevant material from enough evidence to answer	Important diagnostic; a sufficiency judgment can itself be wrong. ^[7]
R1-Searcher, March 2025	Outcome-based RL for search behavior	Emerging; requires a training environment and reliable reward. ^[11]
Search-R1, March 2025	RL with multi-turn retrieval and masking of retrieved tokens	Concrete open research implementation; benchmark-specific improvements do not establish enterprise performance. ^[12]
Why Do Multi-Agent LLM Systems Fail?, March 2025	Failure taxonomy covering design, coordination, and verification	Strong reason to instrument trajectories and test termination. ^[10]

Work and date	Main contribution	Evidence assessment and local consequence
DeepResearcher, April 2025	RL in real web-search environments	Relevant to internet research; the environment differs from a closed local corpus. ^[13]
A-MEM and Mem0, February / April 2025	Structured evolving memory and selective memory extraction	Promising for persistent assistants; add provenance, correction, and deletion before enabling. ^{[16][17]}
HippoRAG 2, February 2025	Graph-based associative retrieval using Personalized PageRank	Emerging alternative for connected facts; evaluate against the same corpus and generator. ^[18]
BrowseComp-Plus, August 2025	Fixed corpus with supporting evidence and hard negatives	Particularly useful for reproducible local search-agent evaluation. ^[14]
Google's agent scaling analysis, January 2026	Controlled comparison of single and multi-agent configurations	Coordination benefit depends on task structure; transfer to local Qwen is unproven. ^[9]
Agentic RAG SoK, March 2026	Formalizes adaptive retrieval as sequential decision making	Conceptual framework; does not supply a production reliability guarantee. ^[15]
Google cross-corpus Agentic RAG, June 5, 2026	Iterative gap detection, routing, and retrieval	Hosted product research; independently implement and evaluate the transferable pattern. ^[8]
Automated prompt injection assessment, June 2026	Evaluates optimized attacks in AgentDojo	Emerging adversarial evidence; testing needs adaptive attacks and utility measurements. ^[20]
BrowseComp-Plus projected onto ClimbMix, August 2026	Tests retrieval with a much larger background corpus	Early evidence of corpus-scale sensitivity; use scaled distractor tests locally. ^[21]

Research on multi-agent scaling should be read narrowly. Google's 180-configuration study used five architectures and proprietary model families across four benchmarks. It reported benefits for parallelizable tasks and degradation for sequential planning; its quantitative gains and predictive accuracy are not transferable hardware-sizing rules. The local recommendation is therefore a centrally controlled single agent with an optional two-worker experiment, not an unrestricted swarm.^[9]

The MAS failure study identified 14 failure modes across system design, inter-agent alignment, and verification/termination in its analyzed systems. It motivates explicit contracts and run traces; it does not establish that a particular framework will fail at a fixed rate. An independent failure-attribution paper also found it difficult to locate the decisive failing step automatically. Retaining structured observations is therefore more useful than relying exclusively on a post-hoc LLM explanation.^{[10][55]}

Search-R1 and R1-Searcher demonstrate that retrieval decisions can be trained with reinforcement learning. They do not imply that local deployment requires RL: prompting an instruction-tuned model through validated tool schemas is the appropriate first implementation. Reproduction requires matching the original search interface, corpus, model checkpoint, reward, and training details; a differently configured local retrieval server is a new experiment.^{[11][12]}

DeepResearcher is valuable evidence that environment choice matters. Its real-web training and browsing components address dynamic pages and noisy search, which a static internal corpus does not reproduce. Importing the architecture without its environment or training should not be presented as reproducing the reported gains.^[13]

BrowseComp-Plus separates the agent from a changing commercial search engine by fixing the corpus. Its original evaluation illustrates that a stronger retriever can improve answers while reducing search calls. A 2026 projection study relocated evidence into a much larger corpus and found sharply reduced evidence recall. The implication is to test both curated evidence and realistic distractors; performance on a small hand-built demo is insufficient.^{[14][21]}

4. The June 2026 Google work

The verified reference is Cyrus Rashtchian and Da-Cheng Juan's June 5, 2026 Google Research article, "Unlocking dependable responses with Gemini Enterprise Agent Platform's Agentic RAG." It describes planning, query rewriting, search fanout, a sufficient-context reviewer, targeted re-search, and synthesis. Evaluation used FramesQA: 824 queries and 2,676 PDF documents. In the cross-corpus experiment, three distracting datasets were added; the article reports 90.1% accuracy, assessed with an LLM judge, and average latency within 3% of the single-corpus system. Its headline "up to 34%" improvement should not be relabeled as 34 percentage points. The article presents a Google-hosted public-preview feature, not a released local implementation.^[8]

The related Sufficient Context paper appeared first in November 2024 and at ICLR 2025. It distinguishes absent evidence from a generator's failure to use available evidence. Its experiments show that sufficient context does not guarantee correct generation, and insufficient context does not preclude a correct answer from prior knowledge. Selective generation trades coverage against correctness; a sufficiency classifier is not a truth oracle.^[7]

Local adaptation, proposed here: represent the question as a short list of answer obligations. Each obligation points to verified evidence spans or is marked missing, ambiguous, or contradicted. Search again only for a concrete remaining gap. Separate draft review from final claim validation. Stop at an explicit budget and return supported partial findings with gaps, or abstain. This is an independently specified design, not a claim of equivalence to Google's internal system.

A simple example is: "Which server does Project Atlas use, and how much memory does it have?" The first authorized document may provide a server ID; a second lookup supplies memory from an inventory table. The answer should cite both sources, preserve their dates, and avoid answering from an unrelated server with a similar name. If inventory is unavailable, the answer can state the server ID and explicitly leave memory unresolved.

5. Framework and implementation landscape

Frameworks organize execution; they do not make a weak model reliable or ensure local operation. Hosted defaults can appear in model clients, embedding helpers, parsing, tracing, task naming, memory extraction, and evaluation. Every dependency must be configured explicitly and tested without external network access.

Framework or implementation	Strength	Local fit and decision
Plain Python state machine	Small dependency surface and explicit control	Valid starting point for a few tools; persistence and recovery must be implemented carefully
LangGraph OSS	State graphs, checkpoints, interrupts, resumable execution	Recommended orchestrator; use local PostgreSQL checkpointing and disable hosted tracing. ^[25]
Haystack	Retrieval pipelines plus an iterative Agent component	Strong alternative if document ingestion/retrieval dominates; requires a tool-capable local chat generator. ^[23]
LlamaIndex / Agent Workflows	Document-oriented components and event-driven async workflows	Good alternative; explicitly select local parsing, embedding, generation, and persistence. ^[24]
Google ADK	Workflow and agent abstractions with local model connectors	Viable alternative; official Ollama integration uses LiteLLM. ADK is distinct from Google's hosted RAG product. ^[22]
Qwen-Agent	Qwen-specific function calling, RAG, and tool integration	Useful reference for model templates; avoid two layers both trying to parse tool output. ^[29]
smolagents	Compact local Transformers support and code-oriented agents	Good learning/research baseline; executing generated code needs a separate sandbox. ^[27]
CrewAI	Role-based crews and event-driven flows	Local models supported; override the documented hosted default and every auxiliary model. ^[28]
AutoGen / Microsoft Agent Framework	Multi-agent research heritage and Python/.NET orchestration	AutoGen is in maintenance mode; assess Agent Framework for a new Microsoft-oriented project. ^[30]
DSPy	Optimizes prompts/programs against a metric	Useful after a stable local evaluation set; not the primary service scheduler or permission system. ^[54]

Framework or implementation	Strength	Local fit and decision
OpenHands	Working software-agent application	Useful separate local coding assistant; its model requirements and sandbox differ from a document assistant. ^[53]
LightRAG	Graph-enhanced retrieval implementation	Optional graph experiment with local backends; not interchangeable with bounded tool orchestration. ^{[19][56]}

Choose one primary orchestrator. Combining LangGraph, CrewAI, ADK, and another workflow engine in the first application creates overlapping ownership of retries, memory, and tool dispatch. For this build, LangGraph owns the reasoning workflow; ordinary Python functions own retrieval and validation; PostgreSQL owns durable records. No cloud orchestration service is required.

Do not confuse checkpoint recovery with exactly-once side effects. A process can fail after a tool commits but before the workflow stores its checkpoint. A durable action ledger, idempotency keys, and reconciliation are required for tools that mutate state. Also bind a saved run to a graph version: LangGraph documentation cautions that changed graph code affects resumed runs.^{[25][57]}

MCP standardizes the exposure of tools and resources. It does not grant permission or prove a tool safe. Tool annotations are untrusted unless the server is trusted; enforce local registration and authorization independently. A2A addresses communication between independent agents. Neither is necessary for internal Python calls on one machine. Add MCP for a specific integration; add A2A only when independent services genuinely need it.^{[43][44]}

6. Models, runtimes, and licensing

Model selection

Use an instruction/post-trained checkpoint, not a base model, for the initial agent. Test structured output, stop behavior, tool selection, argument correctness, evidence grounding, and abstention on the exact quantization. General benchmark rankings are only screening information.

Candidate	Proposed role	Qualification
Qwen3-8B, quantized	Reproducible compatibility baseline	Older 2025 model with documented thinking controls; use to establish the full pipeline. ^[34]
Qwen3.5-9B, quantized	First compact 2026 challenger	Suitable size for interactive evaluation; exact GGUF/runtime/template compatibility must pass tests. ^[33]
Qwen3.8-27B, quantized	Quality challenger on the 64 GB target	Official card available at cutoff; do not assume newer automatically beats smaller models at the workload. ^[32]

Candidate	Proposed role	Qualification
Qwen3.5-27B or 35B-A3B	Alternative quality/MoE experiments	35B-A3B has 35B total parameters despite about 3B active; storage/memory follow total weights. ^[36]
Mistral Small 3.2 24B Instruct	Independent-family generation/judge challenger	Useful to test correlated Qwen errors; preserve its own tokenizer and tool format. ^[37]
Gemma 3 27B instruction model	Optional alternative	Open weights under Gemma terms; not interchangeable with Apache-licensed models. ^[39]

The named Qwen cards identify Apache-2.0 licensing. Mistral's exact checkpoint card should be retained with the artifact. Gemma has a separate license. “Open weights” and “open-source software” are different claims: available inference code or weights do not establish that training data and the full training process are open. Maintain a license record for every model, tokenizer, converter, parser model, dataset, and software package. Do not infer a model's license from the runtime's license.^{[32][33][34][36][37][39]}

No OpenAI-origin model is needed. An OpenAI-compatible HTTP interface is simply a protocol shape served on loopback; it does not imply contacting OpenAI. The proposed application uses a small explicit HTTP adapter and a required local base URL, so no hosted provider key or cloud fallback is part of operation.^[26]

Runtime comparison

llama.cpp is the primary choice because it explicitly supports Apple Silicon through Metal, GGUF quantization, and a local HTTP server. Its server supports schema-constrained responses, but schema validity does not guarantee correct arguments or evidence. The actual schema subset and chat template must be qualified for the pinned build.^[26]

Ollama is a reasonable ease-of-use alternative. Configure its local-only mode, pin imported model artifacts, and verify context and concurrency rather than relying on defaults. Official documentation provides `OLLAMA_NO_CLOUD=1`; an offline test must additionally detect attempts by the surrounding application to call external services.^[41]

MLX-LM is an Apple-focused inference and fine-tuning alternative. Benchmark it against llama.cpp with comparable weights and workload rather than assuming a universal speed advantage. Its own server documentation says the basic server is not recommended for production because of limited security checks; keep it behind the application on loopback if used. Documentation and current implementation differ in breadth, so validate capabilities rather than assuming complete protocol parity.^[40]

vLLM and SGLang are relevant for later accelerator-server deployments and as alternative model-serving implementations. They are not the default Metal runtime for this Mac design. Tool calling depends on model-specific parsers and templates; structured decoding support also varies by mode and release. A future Linux GPU deployment should keep the application contracts and

replace only the inference adapter and capacity profile.^[42]

Run inference natively on macOS. Do not assume that an ordinary Linux Docker container can use the Mac's Metal GPU. Docker's documented Desktop GPU support concerns Windows/WSL2, whereas llama.cpp provides native Metal support. PostgreSQL and CPU services can run in an ARM64 VM/container if desired, but their VM memory still consumes the same 64 GB.^{[26][52]}

Memory and storage planning

These are approximate engineering budgets, not measured M5 performance. Quantized weight bytes are approximately parameters multiplied by bits per weight divided by eight, plus quantization metadata, unquantized tensors, and format overhead. At four bits, raw lower bounds are about 4 GB for 8B, 4.5 GB for 9B, 13.5 GB for 27B, and 17.5 GB for 35B weights, in decimal units. Actual files and runtime residency are larger.

For conventional attention, the KV-cache estimate is $2 \times \text{layers} \times \text{KV_heads} \times \text{head_dimension} \times \text{tokens} \times \text{bytes_per_element} \times \text{concurrent_sequences}$. Hybrid/recurrent architectures require their own cache accounting; do not apply that formula unchanged to all Qwen generations. Measure the loaded model, cache, temporary compute buffers, OS memory pressure, and swap together.

Resource	Initial budget	Operating rule
macOS and ordinary applications	Reserve roughly 12–16 GiB	Leave capacity for the desktop; do not force all memory into inference
Generator and its working buffers	Target at most about 24–30 GiB	One active generation; compact model first, larger model after measurement
Embedding/reranking/parser processes	Target about 2–6 GiB active	Small batches; suspend heavy ingestion during research requests
PostgreSQL, indexes, application	Target about 4–8 GiB	Tune caches conservatively and observe actual resident memory
Remaining headroom	Keep at least about 8 GiB where practical	Reject work or lower context before sustained swap growth

Start with one inference slot and a 16,384-token per-request context cap. This is independent of any advertised maximum. Budget about 8,000 evidence tokens, up to 4,000 for instructions/history/tool definitions, and 2,048 output tokens, retaining slack. Longer research reports should be assembled section by section from a bounded evidence ledger, rather than by keeping the complete run in every prompt.

Do not retain both a large generator and a similarly large judge in memory by default. Run independent-family evaluation offline in a separate window. Multi-agent roles can share the same model process; multiple roles do not require multiple loaded copies, and parallel requests can increase latency on a single memory-bandwidth-limited device.

For 1024-dimensional float32 embeddings, one million vectors contain about 4.096 GB of raw numeric values before row, index, text, and metadata overhead. This arithmetic is a lower bound,

not a database size forecast. Start with a 10,000-document/100,000-chunk pilot ceiling, measure actual expansion, and set quotas. Keep at least 20% of the 2 TB disk free; retain only selected model variants and store encrypted backups on separate media. Local snapshots on the same SSD are not disaster recovery.

7. Retrieval, document processing, and data stores

Use Qwen3-Embedding-0.6B at 1024 dimensions initially. Its official card supports instruction-aware embedding and configurable dimensions; changing dimensions or preprocessing creates a new index generation. Apply the prescribed query instruction and pooling/normalization consistently, and validate vectors against a reference implementation.^[35]

Use Qwen3-Reranker-0.6B as the initial reranker, following its documented input template and scoring method. It is not simply another embedding model or an arbitrary sequence-classification head. Score a small candidate pool and preserve the mapping from each score to the exact chunk. Move to larger embedding or reranking models only after measuring a gain in retrieval and answer quality.^[35]

Docling is the primary parser candidate, with locally prefetched artifacts and remote services disabled. Preserve page identifiers, reading order, headings, table structure, and links back to the immutable source. Scanned PDFs require local OCR assets. Tables and diagrams need a separate validation sample; plain text extraction can silently lose a unit, column header, minus sign, or layout relationship.^[48]

The initial database choice is PostgreSQL with pgvector, ordinary relational metadata, and native text search. This keeps document versions, permissions, full text, vectors, action records, and jobs in one transactional system. pgvector supports exact and approximate search. Begin with exact search for the small pilot, then introduce HNSW only when measured latency requires it.^[38]

PostgreSQL's `ts_rank/ts_rank_cd` lexical retrieval is not BM25. The first hybrid baseline therefore fuses PostgreSQL lexical results and cosine vector results using application-level reciprocal rank fusion. If BM25 is tested, label it as a separate backend/experiment. Compare the whole pipeline rather than calling every lexical implementation equivalent.^[49]

Store alternative	When it helps	Why it is not the initial default
Qdrant	Dedicated vector serving, named dense/sparse vectors, multi-stage retrieval	Adds another service and consistency boundary; adopt when measurement justifies it. ^[50]
SQLite plus a local search/vector extension	Very small single-process prototype	Additional qualification needed for vector extension, concurrent jobs, and durable workflow storage
Dedicated graph store	Traversal over curated relationships	More operational and extraction complexity; first test a small graph in relational tables

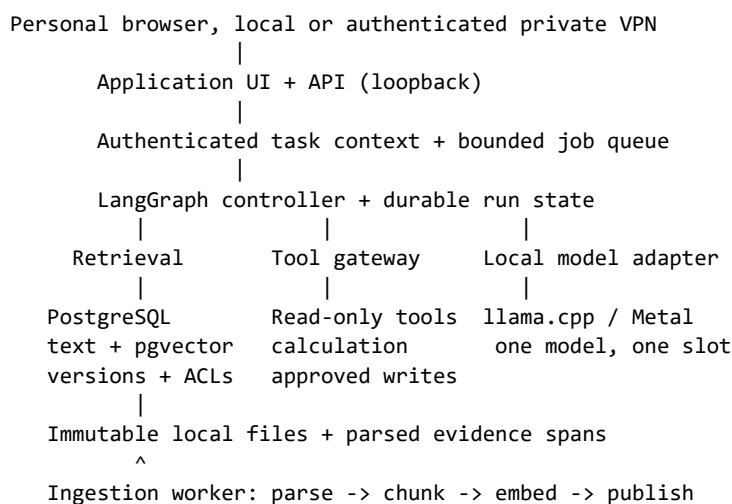
Store alternative	When it helps	Why it is not the initial default
Filesystem object store	Immutable originals and derived parse artifacts	Recommended for the Mac; database stores hashes and metadata, files store bytes

An HNSW filter is not a reason to weaken access control. pgvector documents that approximate scanning followed by filtering can yield too few results; iterative scans or suitable partitioning can improve recall. Keep permission restrictions in the database query and document-fetch path even if a filtered query returns fewer candidates. Never fetch unauthorized chunks and hope the LLM ignores them. ^{[38][51]}

Proposed retrieval sequence: detect exact IDs and collection constraints; issue lexical and vector searches with the same authorization scope and corpus generation; fuse ranks; rerank at most 30 candidates; diversify across documents; expand adjacent sections for selected hits; pack at most 8,000 evidence tokens. The initial values are tunable defaults, not literature-derived optima.

GraphRAG, LightRAG, and HippoRAG 2 are different methods. Global summaries can help “what are the themes?” questions, while relational traversal can help entity chains. Their generated intermediate representations can introduce unsupported edges or lose source nuance. Treat graph-derived answers as hypotheses that must resolve to original spans; deletion must invalidate graph edges and summaries derived from the removed material. ^{[5][18][19]}

8. Reference architecture



Local evaluation + traces -> PostgreSQL / local files

This architecture has separate control and evidence paths. The model sees only authorized evidence and allowed tool schemas. It never receives database credentials, filesystem roots, or authority to alter policy. The tool gateway binds authorization to authenticated application context rather than model-supplied identity fields.

Two modes share the same infrastructure. **Research mode** retrieves, compares, calculates, drafts, and cites. **Task mode** can additionally propose an explicitly supported local side effect. Its action is

prepared as a concrete diff or structured command, then executed only under a matching authorization policy. Arbitrary shell execution and self-modifying agent code are outside the first release.

The controller should produce a structured action such as search, expand, lookup, calculate, or finish. JSON-schema-constrained output is a baseline mechanism; native function calls are an alternative after compatibility testing. Both routes enter the same validator and allowlisted gateway. A model cannot bypass these checks by emitting code in ordinary prose.

Maintain four distinct forms of memory: the current task state; optional conversation summaries; source-backed evidence records; and explicit durable personal preferences. Never promote an unverified generated claim into factual memory. Long-term memory proposals require provenance and can be edited or deleted. This follows the need exposed by recent memory research without assuming that an evolving memory graph is automatically trustworthy.^{[16][17]}

The evidence ledger should store each obligation, candidate claim, supporting source version and span, contradiction, and remaining question. Relevance, sufficiency, factual truth, and permission are separate properties. A document can be relevant but incomplete, sufficient but false, correct but outdated, or informative but unauthorized. Each problem needs a different control.

9. Evaluation and architecture validation

Architecture validation has three levels. **Source-backed feasibility** confirms the components provide the needed local capabilities. **Contract validation** checks interfaces, transitions, budgets, versioning, and data consistency. **Empirical qualification** measures actual model/runtime quality and performance. The first two can be specified now; the third remains a required gate on the incoming Mac. No throughput, latency, or local accuracy figure in this report is represented as measured.

Use a frozen local corpus and a versioned question set. Compare: generator without retrieval; vector-only fixed RAG; hybrid retrieval plus reranking; bounded single-agent retrieval; the same loop without sufficiency assessment; and a limited two-worker variant. Fix model revision, quantization, retrieval assets, prompt versions, token budget, and scoring. Include an oracle-evidence condition to separate retrieval failure from generation failure.

For task mode, grade the actual final tool/database/file state, not just the final message. A fluent “done” can accompany a failed action. For research, grade evidence retrieval, correctness, citation support, coverage, and abstention separately. BFCL offers structured tool-call evaluation; its web-search tracks may need adaptation for an offline setup, so preserve the original benchmark label only when its protocol is unchanged.^[58]

Ragas can provide local model-based metrics when both judge and embedding backends are explicitly configured. Do not accept library defaults or substitute a hosted judge because a tutorial uses one. Calibrate the local judge against human labels and report its disagreement rate. A generator judging itself is correlated evidence, not independent verification.^[59]

Measurement	Definition	Proposed initial release gate
Evidence recall@20	Fraction of annotated required evidence units retrieved in top 20	At least 0.90 on answerable held-out questions
Complete evidence coverage	Fraction of multi-hop questions with all required evidence found	At least 0.80; report separately from mean recall
Answer correctness	Human/adjudicated rubric over all answerable questions; abstentions count as misses	At least 0.85 overall and 0.75 on multi-hop slice
Citation precision	Cited claims actually entailed by the cited span and valid version	At least 0.98
Citation coverage	Externally verifiable answer claims with valid supporting citations	At least 0.95
Appropriate abstention	Unanswerable cases that do not invent the missing answer	At least 0.95
Unsafe execution	Unauthorized mutations or out-of-scope reads actually executed	Zero in the release security suite
Termination	Runs reach a defined terminal state within the hard cap	100% of boundary, timeout, and adversarial tests
Offline integrity	External inference/embedding/judge /parser/telemetry attempts	Zero in the network-isolated functional suite
Agent benefit	Paired gain on multi-hop correctness versus strong fixed RAG	Target at least +5 percentage points, with a positive paired confidence interval

These thresholds are proposed acceptance criteria. They are not expected measurements. If the agent does not clear the benefit gate, keep fixed RAG as the default and expose the adaptive mode as experimental. Report denominator sizes, confidence intervals, run failures, and latency alongside accuracy. Zero observed security failures is not a proof of security.

Create about 300 domain-representative questions once documents are available: 120 for development, 60 for calibration, and 120 locked for final evaluation, split by source document family to reduce leakage. Cover direct facts, exact identifiers, two-to-four-hop questions, time conflicts, tabular arithmetic, no-answer questions, and malicious retrieved content. A 120-question final set gives only a rough estimate; expand slices or the full set when decisions depend on a small difference.

On a stratified subset, repeat each configuration three times. Distinguish first-attempt success from success after retries. Use a paired bootstrap over question IDs, preserving repeated runs within each question; use a binomial interval for absolute success rates. Do not tune thresholds on the held-out test set. Count timeouts and cancellations according to a declared policy rather than

silently omitting them.

Measure time to first progress event, time to final validated answer, retrieval latency, tokens generated, number of searches, peak resident memory, swap growth, parser throughput, and indexing bytes per document. Treat a 90-second interactive and 300-second research deadline as initial product limits, not speed promises. Evaluation should show where a compact model is preferable and where a larger model earns its extra delay.

10. Failure modes and controls

Apply these controls at ingestion, retrieval, tool execution, and answer delivery. Each failure should produce a typed event that can be tested directly, rather than being hidden inside a fluent final response.

Failure	Observable symptom	Required mitigation and test
Retrieved prompt injection	Document asks the agent to ignore policy or transmit data	Treat source content as data; deny network and mutations at gateway; adversarial fixture
Missing second hop	Correct first entity, invented attribute	Obligation ledger and gap-directed search; two-hop fixture
False sufficiency	Reviewer says complete despite a missing clause	Per-obligation evidence validation; calibrate false-positive rate
Citation fabrication	Valid-looking document ID or quote unsupported by source	Server resolves immutable IDs/spans and checks quote correspondence
Temporal conflict	Old policy overrides a newer effective policy	Store effective/publication/ingestion dates separately; report unresolved conflict
Tool-call format error	Invalid JSON, wrong type, nonexistent tool	Constrained schema plus validation; one counted repair, then controlled failure
Repeated search	Same query or same evidence without new progress	Normalized query hashes, evidence delta, explicit budgets
Cross-scope leakage	Search, cache, or citation reveals another collection	Database policy, scoped caches, authorized fetch, revocation test
Retry duplicates	A side effect executes twice after restart	Idempotency ledger and crash-window reconciliation
OCR/table corruption	Wrong unit, row, sign, or heading	Parse QA; retain page image reference and flag uncertain extraction
Memory poisoning	Generated guess becomes a durable “fact”	Provenance, approval policy, expiry, correction, deletion propagation

Failure	Observable symptom	Required mitigation and test
Runaway resource use	Swap growth, thermal throttling, disk exhaustion	One inference slot, backpressure, free-space and memory gates
Hidden cloud dependency	Works online but fails offline or attempts egress	Prefetch assets, explicit adapters, network-isolated integration test
False completion	Final answer says task succeeded but state differs	Verify postconditions and expose failures; never grade prose alone
Upgrade regression	New template breaks tools or checkpoint recovery	Pinned bundles, contract tests, versioned workflows, rollback

AgentDojo shows why normal task accuracy and adversarial robustness need separate measurement. The June 2026 attack study adds evidence that optimized attacks and transfer depend on the attacker and target models. Neither suggests that a prompt-only defense is sufficient. Deterministic permissions and actual execution traces remain the primary enforcement mechanisms.^{[20][60]}

11. Deployment, remote use, and operations

The strict local profile has no internet dependency after assets are staged. Inference, parsing, embedding, reranking, indexing, judging, tracing, and UI assets must all work offline. A connected bootstrap can download verified packages and weights, or an offline transfer can import them. Preserve checksums, licenses, upstream revisions, and a bill of materials. Downloads during installation and hosted inference during operation are different dependencies.

For private remote use, prefer an authenticated VPN terminating at a maintained router or another always-on device. Plain WireGuard can meet the no-hosted-control-plane preference, but needs reachability, key management, and routing. Carrier-grade NAT may require a public endpoint or another networking arrangement. Do not promise access from anywhere without testing the actual ISP and router.^[45]

Tailscale Serve is an optional practical choice for access from enrolled devices. It exposes a local service within the private tailnet; Funnel exposes a service publicly and is not the intended mode here. Tailscale has a hosted coordination dependency even though model inference stays local. Headscale supplies a self-hosted control-server alternative, with its own operational and compatibility burden.^{[46][47]}

Expose only the authenticated application, not the model server, database, metrics, or unrestricted document directories. Enforce owner authentication even when the UI is reached over a VPN. Use secure sessions, origin checks, CSRF protection, upload limits, and rate limits. Trust proxy identity headers only from the configured trusted proxy, never from arbitrary browser requests.

Use macOS service supervision with absolute paths and a dedicated service account. Plan for sleep, reboots, software updates, and disk encryption unlock after power loss. A single Mac remains a single point of failure. Back up source objects and database state consistently, keep an

encrypted copy on separate media, and restore to a fresh directory as a release check. Suggested initial recovery objectives are at most 24 hours of lost updates and recovery within four hours, subject to a measured restore drill.

12. Phased implementation and research roadmap

Phase 0: inventory and qualification. Confirm the actual Mac configuration, macOS version, available memory, disk, and remote-access constraints. Choose a 100-document sample. Freeze one compact generator and its conversion, one runtime, one embedding model, and one reranker. Produce exact artifact hashes and a runtime capability report.

Phase 1: fixed local RAG. Build ingestion, immutable source versions, parsing, chunking, permissions, hybrid search, reranking, and source viewing. Verify offline behavior and retrieval quality before any autonomous loop. Deliver an answer with citations or an explicit no-answer result.

Phase 2: bounded agentic RAG. Add obligations, gap-directed search, evidence expansion, sufficiency assessment, contradiction tracking, and hard budgets. Compare with Phase 1. Retain the simpler default if the measured gain is not meaningful.

Phase 3: general local tools. Add read-only directory lookup and deterministic calculation. Later add narrowly scoped file exports or other explicit operations with previews, authorization, idempotency, and postcondition checks. This phase turns the retrieval assistant into a broader agent without giving it unrestricted operating-system access.

Phase 4: private remote access and recovery. Add the selected VPN profile, responsive UI, owner authentication, backup/restore, service restart, cancellation, and concurrency handling. Prove that database/model endpoints are inaccessible from the remote client.

Phase 5: optimization and research. Compare compact and larger Qwen models, an independent model family, MLX versus llama.cpp, retrieval variants, and a maximum two-worker branch. Only then test graph retrieval, long-term memory, prompt optimization, or local LoRA. RL training remains a separate research project and may require substantially more hardware than inference.

The implementation specification accompanying this report converts these decisions into repository responsibilities, interface contracts, configuration, tests, and stop conditions. It intentionally requires real measurements before declaring the resulting software production-ready. The resulting application can operate without OpenAI, Anthropic, Google, or another hosted model API.

Sources

The following numbered references are the source inventory. Papers are distinguished from vendor reports and living documentation; undated documentation was checked on September 12, 2026. Links identify original sources. Exact deployment pins must be captured during build qualification rather than inferred from a moving documentation URL.

1. Lewis et al.. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#). 2020-05-22. Foundational paper. Accessed September 12, 2026.
2. Yao et al.. [ReAct: Synergizing Reasoning and Acting in Language Models](#). 2022-10-06; ICLR 2023. Research paper. Accessed September 12, 2026.
3. Asai et al.. [Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection](#). 2023-10-17. Research paper. Accessed September 12, 2026.
4. Yan et al.. [Corrective Retrieval Augmented Generation](#). 2024-01-29. Research paper. Accessed September 12, 2026.
5. Edge et al.. [From Local to Global: A Graph RAG Approach to Query-Focused Summarization](#). 2024-04-24; revised 2025-02-19. Research paper. Accessed September 12, 2026.
6. Singh et al.. [Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG](#). 2025-01-15; v4 2026-04-01. Survey. Accessed September 12, 2026.
7. Joren et al.. [Sufficient Context: A New Lens on Retrieval Augmented Generation Systems](#). 2024-11; v3 2025-04-23; ICLR 2025. Research paper; full text consulted. Accessed September 12, 2026.
8. Cyrus Rashtchian and Da-Cheng Juan, Google Research. [Unlocking dependable responses with Gemini Enterprise Agent Platform's Agentic RAG](#). 2026-06-05. Vendor research/product article; full text consulted. Accessed September 12, 2026.
9. Yubin Kim and Xin Liu, Google Research. [Towards a science of scaling agent systems: When and why agent systems work](#). 2026-01-28. Research summary. Accessed September 12, 2026.
10. Cemri et al.. [Why Do Multi-Agent LLM Systems Fail?](#). 2025-03-17. Research paper. Accessed September 12, 2026.
11. Song et al.. [R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning](#). 2025-03-07; v2 2025-03-18. Research paper. Accessed September 12, 2026.
12. Jin et al.. [Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning](#). 2025-03-12; v4 full text consulted. Research paper. Accessed September 12, 2026.
13. Zheng et al.. [DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments](#). 2025-04-04; v4 2025-04-17. Research paper. Accessed September 12, 2026.
14. Chen et al.. [BrowseComp-Plus: A More Fair and Transparent Evaluation Benchmark of Deep-Research Agent](#). 2025-08-08; v1 full text consulted. Benchmark paper. Accessed September 12, 2026.
15. Mishra et al.. [SoK: Agentic Retrieval-Augmented Generation \(RAG\): Taxonomy, Architectures, Evaluation, and Research Directions](#). 2026-03-07. Preprint / systematization. Accessed September 12, 2026.
16. Xu et al.. [A-MEM: Agentic Memory for LLM Agents](#). 2025-02-17; revised 2025-10-08. Research paper. Accessed September 12, 2026.
17. Chhikara et al.. [Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#). 2025-04-28. Author/vendor research paper. Accessed September 12, 2026.
18. Jimenez Gutierrez et al.. [From RAG to Memory: Non-Parametric Continual Learning for Large Language Models](#). 2025-02-20. Research paper. Accessed September 12, 2026.
19. Guo et al.. [LightRAG: Simple and Fast Retrieval-Augmented Generation](#). 2024-10-08; v3 2025-04-28. Research paper. Accessed September 12, 2026.
20. Hofer, DeBenedetti, and Tramer. [Assessing Automated Prompt Injection Attacks in Agentic Environments](#). 2026-06-09. Preprint. Accessed September 12, 2026.

21. Sharifmoghammad et al.. [Projecting BrowseComp-Plus onto ClimbMix: Toward More Realistic Corpora for Agentic Search](#). 2026-08-20. Recent preprint. Accessed September 12, 2026.
22. Google ADK maintainers. [Ollama model host for ADK agents](#). Living documentation. Official documentation. Accessed September 12, 2026.
23. deepset / Haystack maintainers. [Agent](#). Living documentation. Official documentation. Accessed September 12, 2026.
24. LlamaIndex maintainers. [Llama Agents and Agent Workflows](#). Living repository. Official repository. Accessed September 12, 2026.
25. LangChain maintainers. [LangGraph persistence](#). Living documentation. Official documentation. Accessed September 12, 2026.
26. ggml-org maintainers. [llama.cpp and HTTP server documentation](#). Living repository. Official implementation documentation. Accessed September 12, 2026. [Companion source](#).
27. Hugging Face maintainers. [smolagents](#). Living repository. Official repository. Accessed September 12, 2026.
28. CrewAI maintainers. [CrewAI](#). Living repository. Official repository. Accessed September 12, 2026.
29. Qwen maintainers. [Qwen-Agent](#). Living repository. Official repository. Accessed September 12, 2026.
30. Microsoft maintainers. [AutoGen](#). Living repository. Official repository. Accessed September 12, 2026. [Companion source](#).
31. ggml-org maintainers. [llama.cpp](#). Living repository. Official repository. Accessed September 12, 2026.
32. Qwen team. [Qwen3.8-27B model card](#). Living model card; available at cutoff. Developer model card. Accessed September 12, 2026.
33. Qwen team. [Qwen3.5-9B model card](#). Living model card; available at cutoff. Developer model card. Accessed September 12, 2026.
34. Qwen team. [Qwen3-8B model card](#). 2025 model; living card. Developer model card. Accessed September 12, 2026.
35. Qwen team. [Qwen3-Embedding-0.6B and Qwen3-Reranker-0.6B model cards](#). 2025 model series; living cards. Developer model cards. Accessed September 12, 2026. [Companion source](#).
36. Qwen team. [Qwen3.5-35B-A3B model card](#). Living model card; available at cutoff. Developer model card. Accessed September 12, 2026. [Companion source](#).
37. Mistral AI. [Mistral-Small-3.2-24B-Instruct-2506 model card](#). June 2025 model; living card. Developer model card. Accessed September 12, 2026.
38. pgvector maintainers. [pgvector](#). Living repository; page shows v0.8.6 installation. Official documentation. Accessed September 12, 2026.
39. Google. [Gemma 3 27B instruction model card](#). 2025 model; living card. Developer model card. Accessed September 12, 2026.
40. MLX maintainers. [MLX-LM HTTP Model Server](#). Living documentation. Official documentation. Accessed September 12, 2026. [Companion source](#).
41. Ollama maintainers. [FAQ](#). Living documentation. Official documentation. Accessed September 12, 2026.
42. vLLM maintainers. [Tool Calling](#). Living documentation. Official documentation. Accessed September 12, 2026. [Companion source](#).

43. MCP maintainers. [Tools specification](#). 2025-11-25 specification. Protocol specification. Accessed September 12, 2026.
44. A2A Project. [Agent2Agent protocol specification](#). Living specification; pin release when integrating. Protocol specification. Accessed September 12, 2026.
45. WireGuard project. [WireGuard](#). Living official documentation. Official documentation. Accessed September 12, 2026.
46. Tailscale. [tailscale serve command](#). Validated 2026-01-26 on source page. Official documentation. Accessed September 12, 2026. [Companion source](#).
47. Headscale project. [Headscale](#). Living documentation. Official documentation. Accessed September 12, 2026.
48. Docling project. [Advanced options](#). Living documentation. Official documentation. Accessed September 12, 2026. [Companion source](#).
49. PostgreSQL project. [Controlling Text Search](#). PostgreSQL 18 documentation at cutoff. Official documentation. Accessed September 12, 2026.
50. Qdrant. [Hybrid and Multi-Stage Queries](#). Living documentation. Official documentation. Accessed September 12, 2026.
51. PostgreSQL project. [Row Security Policies](#). PostgreSQL 18 documentation at cutoff. Official documentation. Accessed September 12, 2026.
52. Docker. [GPU support in Docker Desktop for Windows](#). Living documentation. Official documentation. Accessed September 12, 2026.
53. OpenHands maintainers. [Run Local LLMs with OpenHands](#). Living documentation; update 2026-05-21 noted. Official documentation. Accessed September 12, 2026.
54. DSPy maintainers. [Language models](#). Living documentation. Official documentation. Accessed September 12, 2026.
55. Zhang et al.. [Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems](#). 2025-04-30. Research paper. Accessed September 12, 2026.
56. HKUDS maintainers. [LightRAG API Server](#). Living documentation. Official implementation documentation. Accessed September 12, 2026.
57. LangChain maintainers. [LangGraph backward compatibility](#). Living documentation. Official documentation. Accessed September 12, 2026.
58. Berkeley / Gorilla project. [BFCL V4: Web Search](#). 2025-07-17; updated 2025-07-20. Benchmark documentation. Accessed September 12, 2026.
59. Ragas maintainers. [RAG Evaluation Quickstart](#). Living documentation. Official documentation. Accessed September 12, 2026.
60. DeBenedetti et al.. [AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents](#). 2024-06-19. Research paper. Accessed September 12, 2026.